

Artificial Intelligence

CSE 4101

Department of Computer Science and Engineering
University of Dhaka

Lecture : Decision Making in AI

Decision Theoretic Agent

- Decision-theoretic agent—an agent that can make rational decisions based on what it believes and what it wants.

“How utility theory combines with probability theory to yield a decision-theoretic agent?”

- Such an agent can make decisions in contexts in which uncertainty and conflicting goals leave a logical agent with no way to decide.
- a goal-based agent has a binary distinction between good (goal) and bad (non-goal) states, while a decision-theoretic agent has a continuous measure of outcome quality...

Utility Theory

- an agent's preferences are captured by a ***utility function***.
- This function provides a map from the states of the world to a real number.
- The bigger the number the more the agent likes that particular state.

$$u_i : S \rightarrow \mathbb{R}$$

- the **states** are defined as **those states of the world** that the agent can perceive.
- a robot has a utility function defined over **only two states** regardless of how complicated the world is- if a robot has only one sensor that feeds him a binary input.
 - Bright 1 and 0 if its dark, $u_i(0) = 5$, $u_i(1) = 10$
- In practice, however, agents have sophisticated inputs, and it is impractical to define a different output for each input.
- mapping raw inputs to a smaller set of world states!!!

Preference Ordering

- an agent's preferences are captured by a ***utility function***.
- Given an agent's utility function we can define a preference ordering for the agent over the states of the world.
- By comparing the utility values of two states we can determine which one is preferred by the agent.

- *reflexive*: $u_i(s) \geq u_i(s)$
- *transitive*: If $u_i(a) \geq u_i(b)$ and $u_i(b) \geq u_i(c)$ then $u_i(a) \geq u_i(c)$
- *comparable*: $\forall_{a,b}$ either $u_i(a) \geq u_i(b)$ or $u_i(b) \geq u_i(a)$.

Preference Ordering

Completeness -

- Dave strictly prefers larger chocolate bars to smaller ones (Complete!)
- Susie says that she always prefers the bigger and darker chocolate bar (Incomplete!)

Reflexivity

- Relationships are reflexive if they can be applied when both sides of the relationship are the same
- I am at least as old as myself

Greater than or equal to ($\geq$):

- A number is greater than or equal to itself.

Example: $3 \geq 3$

Uses of Utility Function

- We can use utility functions to describe the behavior of almost every agent.
- useful for capturing the various tradeoffs that an agent must make, along with the value or expected value of its actions.
 - *package delivery agent – payment, electricity consumption cost and*
 - *opportunity cost* - the **value of foregone alternatives**, an indirect cost.
- Once we have defined a utility function for all the agents then all they have to do is take actions which maximize their utility.

Does utility correspond to money ?

- Bill has \$100 million while Tim has \$0 in the bank.
- They both contemplate the possibility of winning one million dollars.
- that extra million will make a significant difference in Tim's lifestyle while Bill's lifestyle will remain largely unchanged.
- Tim's utility for the same million dollars is much larger than Bill's.
- Notice that we are considering the **marginal utility** of money, that is, the utility for the next million dollars. -
- assume that both Bill and Tim have the same utility for their first million dollars.

Marginal utility

- Zero Marginal Utility - additional newspaper of same edition
- Positive Marginal Utility - A coupon for an additional hair-cut
- Negative Marginal Utility – Antibiotics

- Increasing Marginal Utility – Finding the second shoe
- Diminishing Marginal Utility - Holding your breath under water – Second breath

Framing Effect

Framing Effect refers to the psychological phenomenon where individuals make different decisions depending on how identical information is presented — as a gain or as a loss

People's decisions are influenced by how information is presented—**gains** vs. **losses**—even when the underlying facts remain the same.

Scenario 1: Gain Frame

You receive \$1000. Choose one:

- Option A: Keep \$500
- Option B: 50% chance to keep \$1000, 50% chance to keep nothing

Most people choose Option A (risk-averse)

Scenario 2: Loss Frame

You receive \$1000. Choose one:

- Option A: Lose \$500
- Option B: 50% chance to lose \$1000, 50% chance to lose nothing

Most people choose Option B (risk-seeking)

- Even when the expected outcomes are identical, the **framing** of choices can significantly alter decisions.
- This has important implications in fields such as **behavioural economics, decision theory, AI planning, and policy-making**.

Expected Utility

- each agent has some sensors which it can use to determine the state of the world and some actuators which it can use to take action.
- These actions can lead to new states of the world.
- For example, an agent senses its location and decides to move forward one foot. Of course, these sensors and actuators might not operate perfectly:
 - the agent might not move exactly one foot or its sensors might be noisy.
- Let's assume, however, that the agent does know the probability of reaching state s' given that it is in state s and takes action a .

This probability is given by $T(s, a, s')$ which we call the transition function.

Expected Utility (cont.)

- Since the transition function returns a probability then it must be true that the sum of $T(s, a, s')$ over all possible a and s' is equal to one.
- Using this transition function the agent can then calculate its expected utility expected utility for taking action a in state s as:
- Here S is the set of all possible states.

$$E[u_i, s, a] = \sum_{s' \in S} T(s, a, s') u_i(s'),$$

The Value of Information

- consider a piece of new information that leads the agent to determine that it is not really in state **s** but it is instead in state **t**.

$$E[u_i, s, a] = \sum_{s' \in S} T(s, a, s') u_i(s'),$$

$$E[u_i, t, \pi_i(t)] - E[u_i, t, \pi_i(s)].$$

- The agent's behavior is captured by a **policy** which is a mapping from **states to actions**. We will use **π** to denote the agent's policy.
- The value of the piece of information is the expected utility the agent receives now that he knows he is in **t** and takes action accordingly minus the utility it would have received if he had instead mistakenly assumed it was in **s** and taken action accordingly.

The Value of Information - Applications

$$E[u_i, t, \pi_i(t)] - E[u_i, t, \pi_i(s)].$$

- doctor has made a preliminary decision about which drug to prescribe for a patient but is uncertain about the patient's specific disease.
- She can calculate the value of information acquired from performing various tests which could identify the specific disease.
- If a test, regardless of its outcome, would still lead the doctor to prescribe the same drug then the value of the information from that test's result is zero.
- On the other hand, a test that is likely to change the doctor's action will likely have a high value of information.

The maximum expected utility

- The agent's goal then becomes that of finding its optimal policy which is the one that *maximizes its expected utility*, as we might expect a “selfish agent” to do.
- This strategy is known as the principle of **maximum expected utility**.

$$\pi_i^*(s) = \arg \max_{a \in A} E[u_i, s, a].$$

Decision Networks / Influence Diagram

- decision network represents information about the agent's current state, its possible actions, the state that will result from the agent's action, and the utility of that state.
 - *Decision node/action* nodes (corresponding to each decision to be made) is drawn as a rectangle.
 - *Uncertainty/chance nodes* (corresponding to each uncertainty to be modelled) is drawn as an oval.
 - *Value/Utility nodes* is drawn as an octagon (or diamond).

Example – Oil Wild-catter Problem

- Assume 3 states of nature: $\mathbf{o} = 0$ (well is dry), $\mathbf{o} = 1$ (has some oil) and $\mathbf{o} = 2$ (soaking)
- your prior beliefs are $\mathbf{p(o)} = [0.5, 0.3, 0.2]$.
- Two possible actions: $\mathbf{d} = 0$ (don't drill) and $\mathbf{d} = 1$ (drill)

	$o = 0$	$o = 1$	$o = 2$
$d = 0$	0	0	0
$d = 1$	-70	50	200

$$EU(d = 1) = \sum_{o=0}^2 p(o)U(d, o) = 0.5 \cdot (-70) + 0.3 \cdot 50 + 0.2 \cdot 200 = 20$$

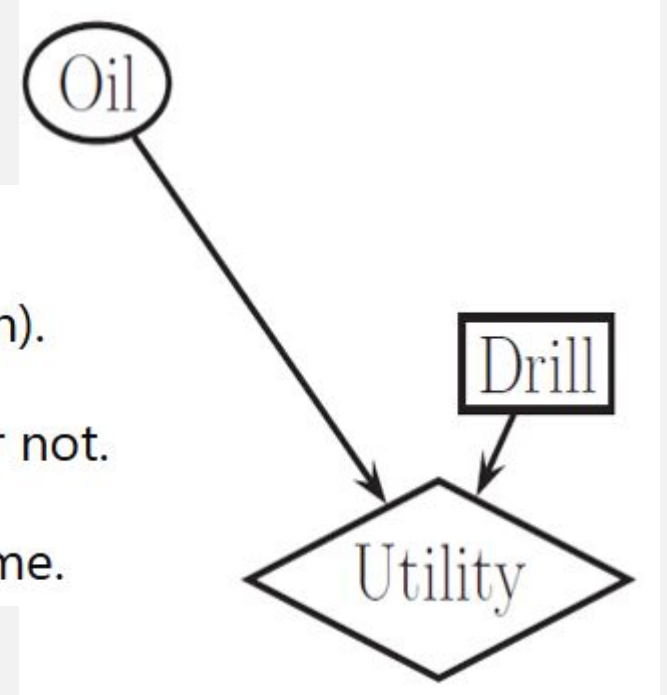
$$MEU = \max\{EU(d = 0), EU(d = 1)\} = \max\{0, 20\} = 20$$

$$d^* = \arg \max\{EU(d = 0), EU(d = 1)\} = 1$$

Influence/Decision Diagram of Oil Wild-catter Problem

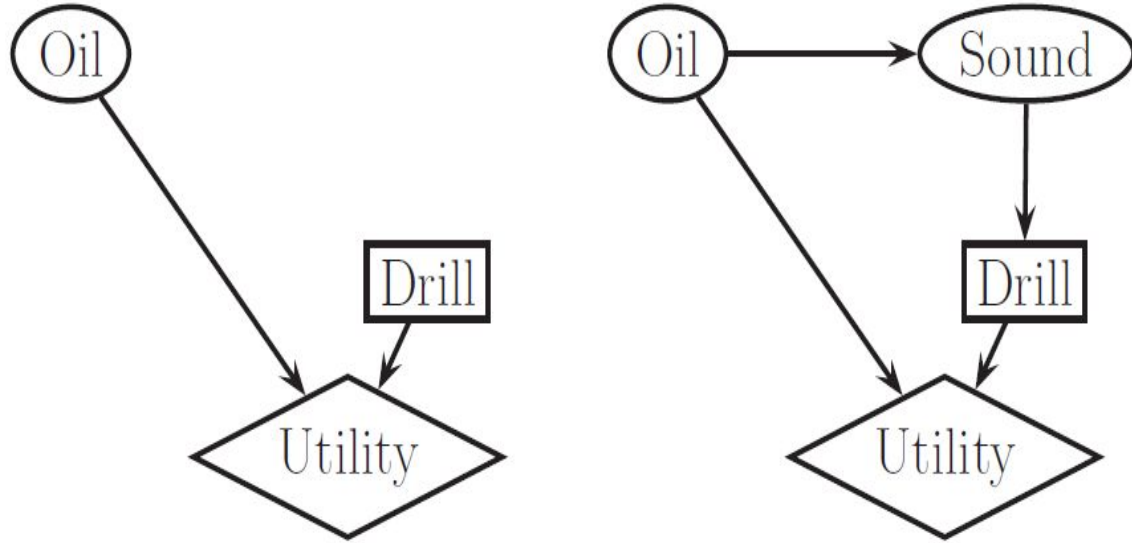
Nodes:

- **Oil** (oval): A **chance node** representing whether the site has oil (uncertain).
- **Drill** (square): A **decision node** representing the agent's choice to drill or not.
- **Utility** (diamond): A **value node** representing the payoff from the outcome.



- Oil → Utility: The presence of oil affects the final utility.
- Drill → Utility: The choice to drill also affects the utility.

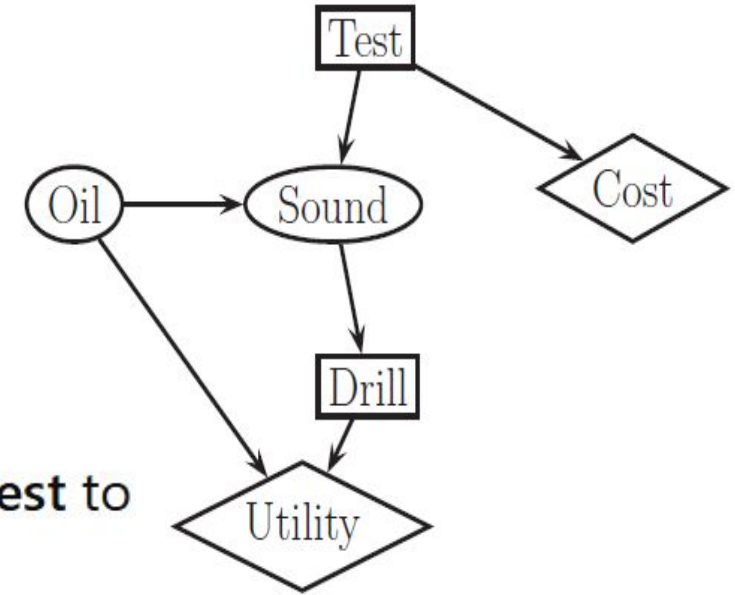
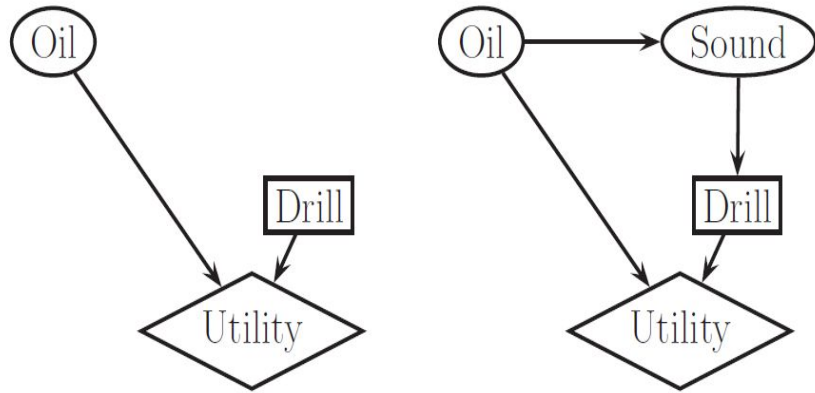
Influence/Decision Diagram of Oil Wild-catter Problem



Adds a new variable: **Sound** (oval), which represents a signal or clue about the presence of oil (e.g., a seismic test result or geologist's report).

- Oil → Sound: The sound is generated **based on the presence of oil**.
- Sound → Drill: The agent **observes** the sound before making the drill decision.
- Oil, Drill → Utility: Final utility still depends on the true oil state and the decision.

Influence/Decision Diagram of Oil Wild-catter Problem



Test (decision node): The agent decides **whether or not to run a costly test** to generate the Sound observation.

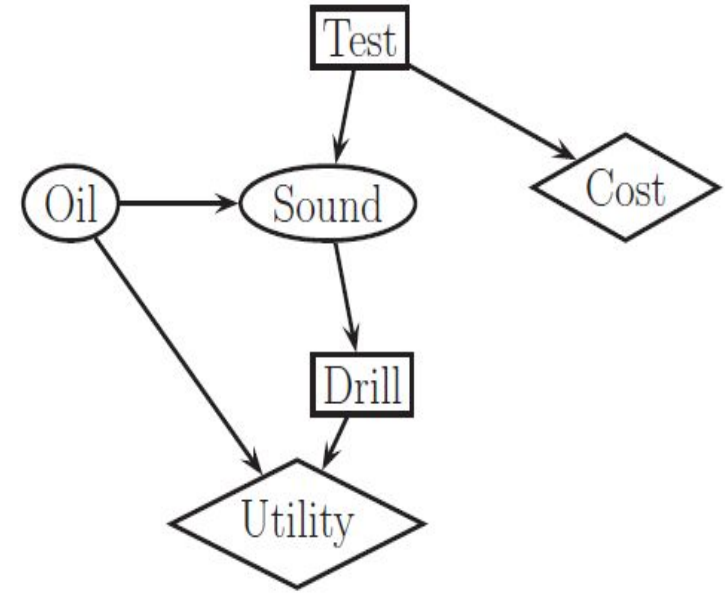
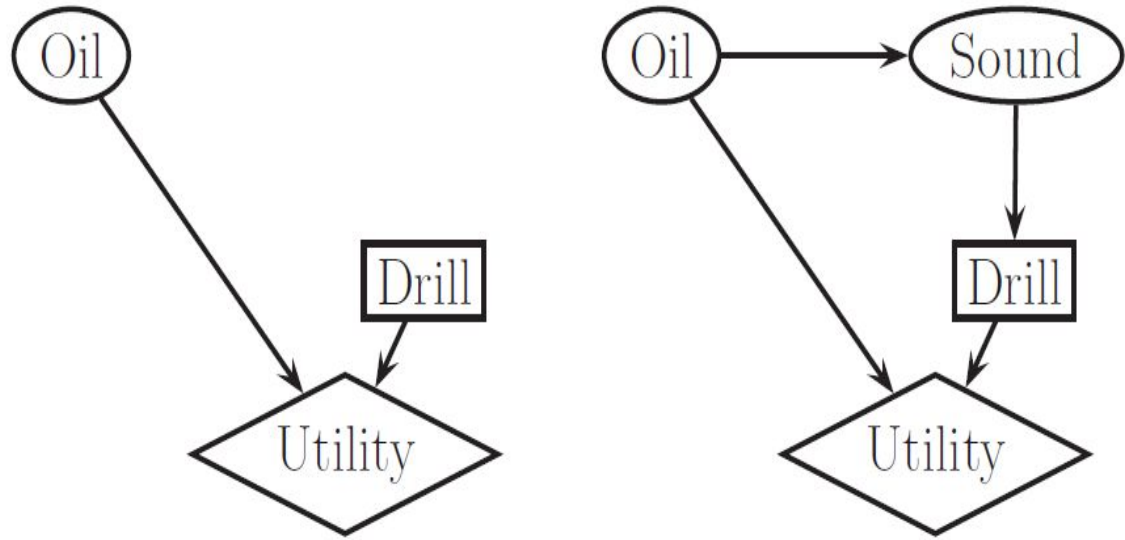
Cost (value node): Represents the **cost incurred** if the test is performed.

Test → Sound: The sound is only generated **if the test is run**.

Test → Cost: Cost is applied depending on the decision.

The rest remain similar to the middle diagram.

Influence/Decision Diagram of Oil Wild-catter Problem



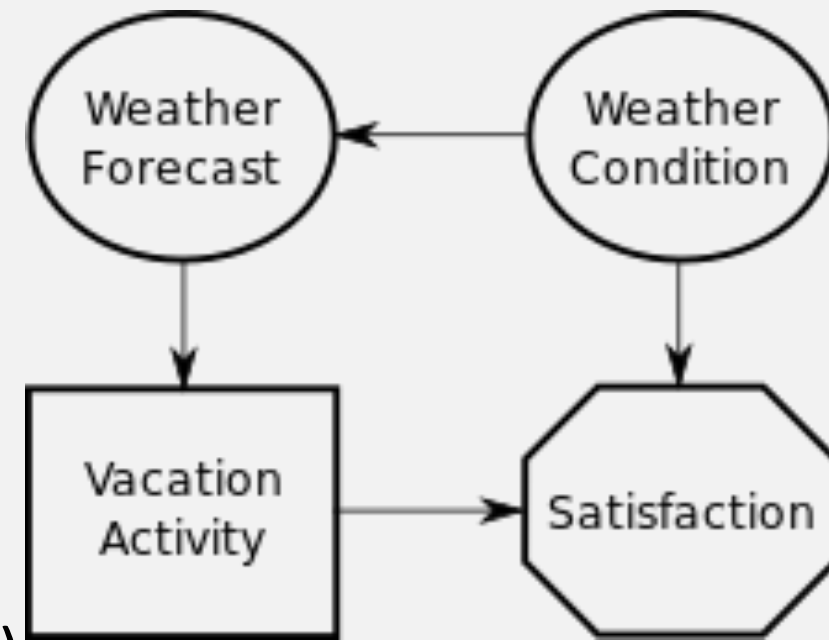
	$s = 0$	$s = 1$	$s = 2$
$o = 0$	0.6	0.3	0.1
$o = 1$	0.3	0.4	0.3
$o = 2$	0.1	0.4	0.5

$$p(o|s = 0)$$

$$EU(d|s = 0) = \sum_{o=0}^2 p(o|s = 0)U(o, d)$$

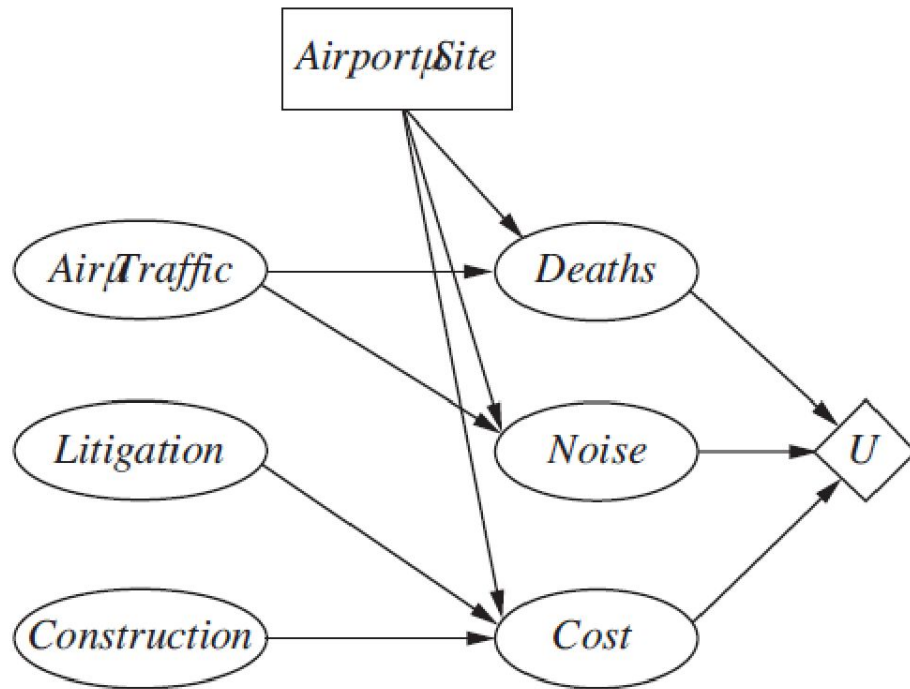
Decision Networks / Influence Diagram

- *Decision /action node (rectangle)*
- *Uncertainty/ chance node (Oval)*
- *Value node/Utility nodes (diamond/octagon).*
- There is 1 conditional arc (ending in *Weather Forecast*)
 - Conditional arc ending in *Weather Forecast* indicates their belief that *Weather Forecast* and *Weather Condition* can be dependent.
- 1 informational arc (ending in *Vacation Activity*).
 - Informational arc ending in *Vacation Activity* indicates that they will only know *Weather Forecast*, not *Weather Condition*, when making their choice.
- 2 functional arcs (ending in *Satisfaction*)
 - Functional arcs ending in *Satisfaction* indicate that *Satisfaction* is a utility function of *Weather Condition* and *Vacation Activity*.



Decision Networks / Influence Diagram

- *Decision node (rectangle)*
- *Uncertainty/ node (Oval)*
- *Value node/Utility nodes (diamond/octagon).*



A simple decision network for the airport-siting problem.

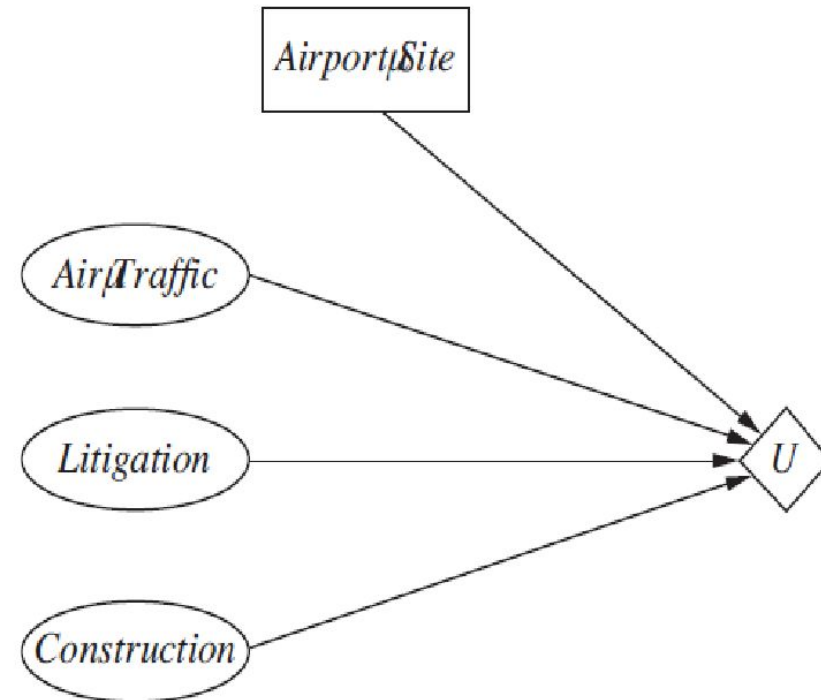


Figure 16.7 A simplified representation of the airport-siting problem. Chance nodes corresponding to outcome states have been factored out.

Evaluating Decision Networks / Influence Diagram

- Actions are selected by evaluating the decision network for each possible setting of the decision node.
- Once the decision node is set, it behaves exactly like a chance node that has been set as an evidence variable.

1. Set the evidence variables for the current state.
2. For each possible value of the decision node:
 - (a) Set the decision node to that value.
 - (b) Calculate the posterior probabilities for the parent nodes of the utility node, using a standard probabilistic inference algorithm.
 - (c) Calculate the resulting utility for the action.
3. Return the action with the highest utility.

Markov Decision processes

Definition 1.1 (Markov Decision Processess). An MDP consists of an initial state s_1 taken from a set of states S , a transition function $T(s, a, s')$ and a reward function $r : S \rightarrow \mathbb{R}$.

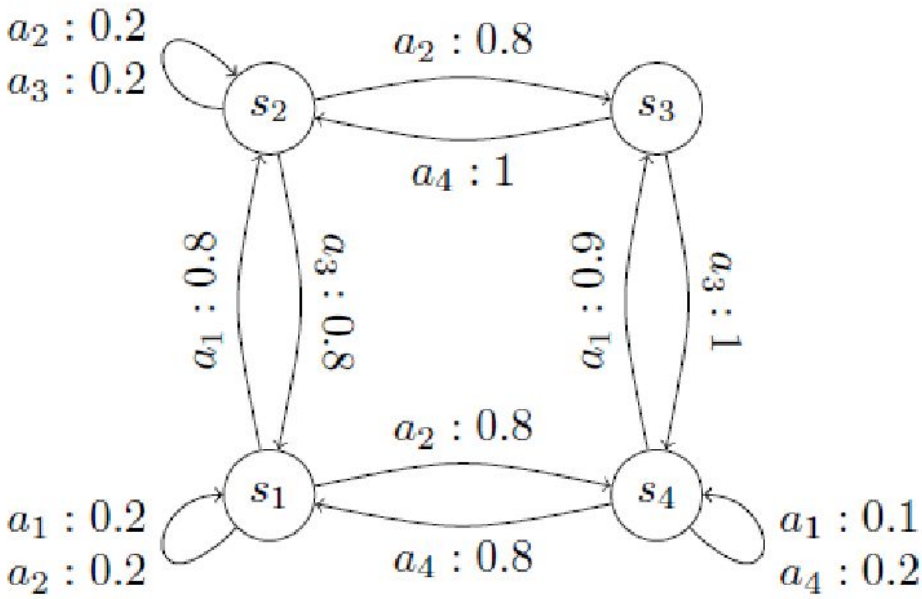
optimal **policy** leads to the **maximum expected utility**

$$\pi_i^*(s) = \arg \max_{a \in A} E[u_i, s, a].$$

Where -

$$E[u_i, s, a] = \sum_{s' \in S} T(s, a, s') u_i(s'),$$

Figure 1.1: Graphical representation of a sample Markov decision process along with values for the transition and reward functions. We let the start state be s_1 .



s	$r(s)$
s_1	0
s_2	0
s_3	1
s_4	0

s_i	a	s_j	$T(s_i, a, s_j)$
s_1	a_1	s_1	0.2
s_1	a_1	s_2	0.8
s_1	a_2	s_1	0.2
s_1	a_2	s_4	0.8
s_2	a_2	s_2	0.2
s_2	a_2	s_3	0.8
s_2	a_3	s_2	0.2
s_2	a_3	s_1	0.8
s_3	a_4	s_2	1
s_3	a_3	s_4	1
s_4	a_1	s_4	0.1
s_4	a_1	s_3	0.9
s_4	a_4	s_4	0.2
s_4	a_4	s_1	0.8

Markov Property

A process satisfies the **Markov Property** if the **next state depends only on the current state** — not on the sequence of events that preceded it.

$$P(S_{t+1} \mid S_t, S_{t-1}, \dots, S_0) = P(S_{t+1} \mid S_t)$$

- **Memoryless Property:** The system has no “memory” of the past.
- **Simplifies Modelling:** We only need the current state to make predictions or decisions.

Why is this relevant to MDPs?

- **Efficient Computation:** No need to store or process full histories.
- **Mathematical Tractability:** Enables recursive algorithms (e.g., Value Iteration).
- **Foundation for Reinforcement Learning:** Most RL models assume environments are Markovian.

Policy to deal with future reward possibility

The Policy Question

- Should an agent take **100 actions with no reward** to reach a state with a **reward of 100**?
- Or take **100 actions**, each giving a **reward of 1**?

Agents don't live forever — waiting too long might not pay off.

Immediate rewards are typically **safer and more reliable**.

Delaying gratification can be smart — but only to a point.

Policy to deal with future reward possibility (cont.)

- **Discounted rewards** allow us to smoothly reduce the impact discounted rewards of rewards that are farther in the future.

- Multiply future rewards by a factor γ^n , where $0 < \gamma < 1$.
- This gives less weight to rewards further in the future.

$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') u(s'),$$

$$\text{Discounted return: } \gamma^0 r(s_1) + \gamma^1 r(s_2) + \gamma^2 r(s_3) + \dots$$

- we can define the real utility the agent receives for being in state s as:

$$u(s) = r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u(s')$$

- This is known as the **Bellman equation**, and it captures the fact that an agent's utility depends not only on its immediate rewards but also on its future discounted Bellman equation rewards.

Generate Utility Function

$$u(s) = r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u(s')$$

- The problem we now face is how to calculate $u(s)$ given the MDP definition.
- One approach is to solve the set of equations formed.
- Given n states we have n Bellman equations each one with a different variable.
- We thus have a system of n equations with n variables so, theoretically, we can find values for all these variables.
- In practice, however, solving this set of equations is not easy because of the **max operator** in the Bellman equation which makes the **equations non-linear**.

Value Iteration, Bellman Equation and Bellman Update Equation

$$u(s) = r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u(s')$$

Value Iteration: we start by setting the values of $u(s)$ to some arbitrary numbers and then **iteratively** improve these numbers using the **Bellman update equation**:

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s')$$

we are now **updating the u values over time**. It has been shown that this process will eventually, and often rapidly, **converge to the real values of $u(s)$** .

Value Iteration, Bellman Equation and Bellman Update Equation

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s')$$

if the maximum change in utility for a particular time step is less than $\epsilon(1 - \gamma)/\gamma$ then the error is less than ϵ . This fact can be used as a stopping condition.

Relationship to Error (ϵ)

- ϵ is a user-defined error threshold.
- It controls **how accurate** the final utility estimates should be.

Smaller ϵ implies:

- The algorithm takes **longer** to converge.
- The final result is **more precise**.

Value Iteration, Bellman Equation and Bellman Update Equation

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s')$$

if the maximum change in utility for a particular time step is less than $\epsilon(1 - \gamma)/\gamma$ then the error is less than ϵ . This fact can be used as a stopping condition.

The Role of the Discount Factor (γ)

The discount factor (γ) controls how much we care about future rewards compared to immediate rewards. Its interaction with ϵ is important:

- **Higher γ :** When γ is close to 1, we strongly consider future rewards. This generally makes convergence slower, as a tiny change in a distant state can slightly influence the current iteration. Consequently, a smaller ϵ is needed to compensate for accuracy.
- **Lower γ :** When γ is much smaller than 1, we prioritize immediate rewards. Convergence may be faster, as distant state values have less influence.

Stopping Criterion

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s')$$

if the maximum change in utility for a particular time step is less than $\epsilon(1 - \gamma)/\gamma$ then the error is less than ϵ . This fact can be used as a stopping condition.

While the stopping criteria sound straightforward in theory, there are nuances to keep in mind:

- **No guarantees:** The utility change criterion doesn't guarantee finding absolutely optimal values. It mainly signals that further iterations are unlikely to yield significantly better results.
- **Local optima:** The algorithm might get stuck in a local optimum, where values seem stable but aren't truly the best possible solution.
- **Other criteria:** Often, in practice, multiple stopping criteria are used in conjunction. For example:
 - A maximum number of iterations to set a time limit.
 - Performance monitoring in the actual environment, if applicable.

Value Iteration Algorithm

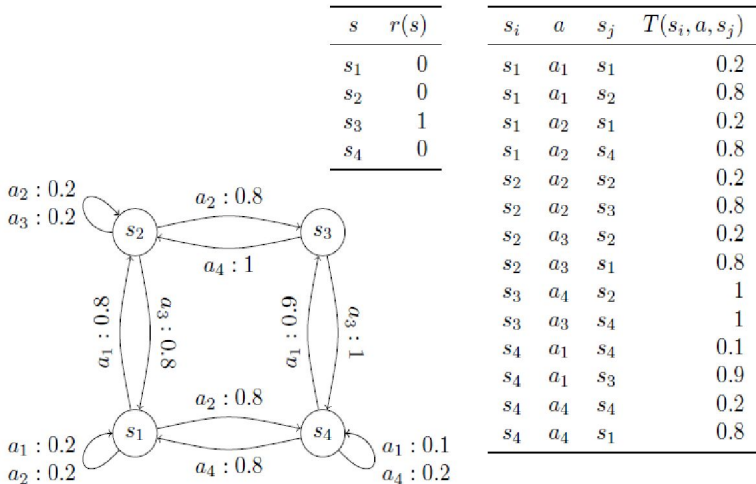
	Time (t)				
	0	1	2	3	4
$u(s_1)$	0	0	0	$.5(.8).45 = .18$	$.5(.036 + .376) = .206$
$u(s_2)$	0	0	$.5(.8)1 = .4$	$.5(.88)1 = .44$	$.5(.088 + .96) = .52$
$u(s_3)$	0	1	1	$1 + .5(1).45 = 1.2$	$1 + .5(.47) = 1.2$
$u(s_4)$	0	0	$.5(.9)1 = .45$	$.5(.9 + .045) = .47$	$.5(1.1 + .047) = .57$

s	$r(s)$
s_1	0
s_2	0
s_3	1
s_4	0

s	$\pi^*(s)$
s_1	a_2
s_2	a_2
s_3	a_3
s_4	a_1

VALUE-ITERATION(T, r, γ, ϵ)

```
1      do
2           $u \leftarrow u'$ 
3           $\delta \leftarrow 0$ 
4          for  $s \in S$ 
5              do  $u'(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s')u(s')$ 
6                  if  $|u'(s) - u(s)| > \delta$ 
7                      then  $\delta \leftarrow |u'(s) - u(s)|$ 
8          until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
9  return  $u$ 
```



Markov Decision Processes

Markov Decision Processes (MDPs) serve as the backbone of both model-free and model-based reinforcement learning (RL).

- Both model-free and model-based RL methods aim to solve problems that can be modeled as MDPs.
 - This means they deal with environments where an agent interacts with the world in discrete time steps, receiving rewards and transitioning between states.
 - The core elements of an **MDP (states, actions, transition functions, rewards)** are fundamental in both types of RL algorithms.

Markov Decision Processes

Markov Decision Processes (MDPs) serve as the backbone of both model-free and model-based reinforcement learning (RL).

Model-Based RL

- **Learning a model:** Model-based RL algorithms first attempt to **learn a model of the environment** – how the environment responds to the agent's actions, and the rewards it generates.
- **Planning:** Once the model is learned (or approximated), these algorithms use planning techniques (like **value iteration or policy iteration**) on the learned model to find an optimal policy.

Markov Decision Processes

Markov Decision Processes (MDPs) serve as the backbone of both model-free and model-based reinforcement learning (RL).

Model-Free RL

- **Learning without a model:** Model-free RL algorithms (like Q-learning, SARSA) learn directly from experience without explicitly learning a model of the environment's dynamics (i.e., the transition function and reward function).
- **Focus on Values:** They concentrate on learning value functions (such as, Q-values) that tell the expected future reward based on a **state-action pair**.
- **Trial and error:** Through repeated interaction, the agent gradually improves its policy (its "strategy" for taking actions).

Markov Decision Processes – Key Trade-offs

Sample efficiency vs. Computation

- Model-free methods tend to be **more sample-efficient (they need less experience to learn)** but can be computationally expensive.
- Model-based methods can be more computationally efficient for planning but might require more samples/experience to learn a good model.

Flexibility

- Model-free methods are often more adaptable to changes in the environment, as they don't rely on an explicit model.
- Model-based methods might need to re-learn their model if the environment changes significantly.

In Summary

- MDPs provide the theoretical framework in which both model-free and model-based RL operate.
- The distinction lies in whether an algorithm explicitly attempts to learn the ***inner workings of the environment (model-based)*** or focuses on learning values and policies ***directly from experience (model-free)***.

Multiagent Markov Decision Processes

- The easiest way - place all the other agents' effects into the transition function.
 - assume the other agents don't really exist as entities and are merely part of the environment
 - This technique can work for simple cases (**rare!**) where the agents are not changing their behaviour since the transition function in an MDP must be fixed.
- **A better method** - extend the definition of MDP to include multiple agents all of which can take an action at each time step.

$$T(s, \vec{a}, s')$$

- vector of size equal to the number of agents - where each element is an agent's action or a symbol representing non-action by that agent.
- **How to define the reward** : divide evenly ? or give each agent a reward proportional to his contribution to the system's reward.

Reinforcement Learning

- In machine learning, a common drawback is the large amount of data that models need to train. The more complex a model, the more data it may require.
- the data we get may not be reliable. It may have false or missing values or may be collected from untrustworthy sources.
- **Reinforcement learning** is a branch of Machine Learning that trains a model to come to an optimum solution for a problem by taking decisions by itself
 - Overcome the issue of data collection by almost completely removing the need for data.
- An Environment, which an agent will interact with, to learn to reach a goal or perform an action.
- A Reward if the action performed by the model is bringing us closer to the goal/is leading to the goal. This is done to train the model in the right direction.
- A negative reward if it performs an action that will not lead to the goal to prevent it from learning in the wrong direction.

Reinforcement Learning - Classifications

Model-Based vs. Model-Free

Model-Based

The agent learns or has access to a model of the environment (how it **transitions between states** and **gives rewards**).

- It uses this model for planning.
- Example: Value iteration and Policy iteration algorithms

Model-Free

- The agent doesn't explicitly learn a model of the environment.
- It learns directly from experience and trial-and-error.
 - Examples: Q-learning, SARSA, Deep Q-Networks (DQN)

Reinforcement Learning - Classifications

Value-Based vs. Policy-Based

Value-Based

- Focus on learning the value of being in a state (state-value) or taking an action in a state (action-value).
- The optimal policy is indirectly derived from these values.
- Examples: Q-learning, Deep Q-Networks (DQN)

Policy-Based

- Learn the policy (that maps states to actions) directly without using an explicit value function.
- Represent the policy explicitly,
- typically, parameterized - Optimizes the policy parameters to maximize expected reward.
- Examples: REINFORCE, Actor-Critic methods, PPO

Reinforcement Learning - Classifications

On-Policy vs. Off-Policy

- **On-Policy:** The agent learns from data generated by the current version of a policy that it's actively optimizing.
- The policy being improved is the **same policy** used to make decisions and gather experiences.

Example: SARSA

- **Off-Policy:** Learns the optimal policy (i.e. target policy) while using a different, often more exploratory, policy (behaviour policy) to generate data.
 - Suppose a robot collected data by randomly exploring a maze (behavior policy: random policy).
 - Later, it uses these stored experiences to train and improve a smarter, goal-directed policy (target policy: learned optimal policy).
- Example: Q-learning

Q-learning – Off-policy model free RL

“Q” stands for quality -represents how valuable the action is in maximizing future rewards, in other words, the quality of a particular action taken from a particular state...

Objective:

- Determines the best action based on the current state of the agent.
- May establish its own rules or explore beyond given policies.

Off-policy nature:

- Doesn't require a predefined policy to follow.
- Can operate outside given policies to find optimal actions.

Model-free approach:

- Relies on predictions of the environment's response to move forward.
- Learns through trial and error rather than explicit reward guidance.

An example of Q-learning is an advertisement recommendation system

Q-learning – Off-policy model free RL

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- $Q(s, a)$:

The current Q-value of taking action a in state s . Represents the agent's estimate of how good this action is.

- α (learning rate):

Controls how much new information overrides old values. Ranges between 0 and 1.

- r (reward):

The immediate reward **observed** after taking action a in state s , resulting in transition to next state s' .

(Note: This is an observed reward — not learned or predicted by a model.)

- γ (discount factor):

Determines how much future rewards are valued compared to immediate ones. Value in $[0, 1]$.

- $\max_{a'} Q(s', a')$:

The estimated value of the **best action** at the next state s' . This makes Q-learning **off-policy**, since it uses a greedy target policy even if behavior was exploratory.

Episodes: the end of the stage, where agents can't take new action. It happens when the agent has achieved the goal or failed.

Q-learning – Off-policy model free RL

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$$u^{t+1}(s) \leftarrow r(s) + \gamma \max_a \sum_{s'} T(s, a, s') u^t(s')$$

Requires a full model: The transition function $T(s, a, s')$ must be known.

Used in planning algorithms like Value Iteration or Policy Iteration.

Q-learning uses the observed reward r and next state s' to update $Q(s, a)$, rather than relying on known transition probabilities $T(s, a, s')$.

It estimates value **from actual experience**, not through planning with a model.

Q-learning is a form of **TD learning**:

- It updates Q-values based on the **difference between the predicted Q-value and the observed reward + estimated future return**.
- Blends advantages of **Monte Carlo** (learning from actual experience) and **Dynamic Programming**

Q-learning – Off-policy model free RL

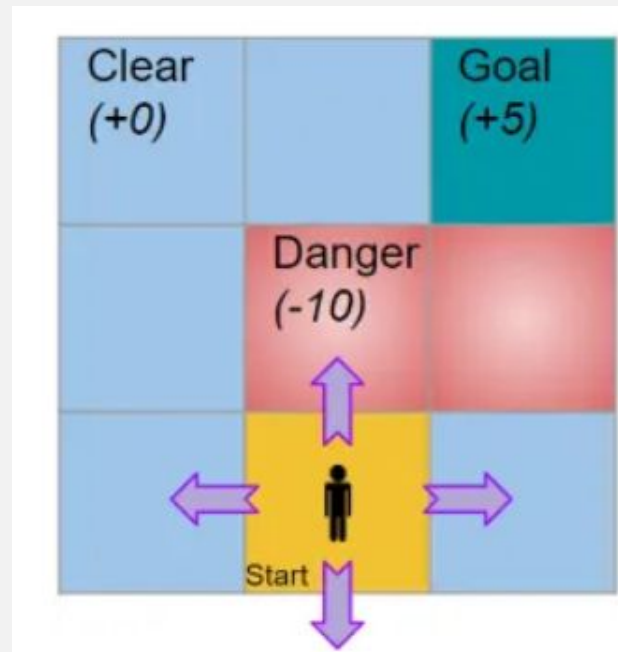
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- ❑ This problem has 9 states since the player can be positioned in any of the 9 squares of the grid. It has 4 actions.

Cell value (3, 2) is the Q-value (or State-Action value)

for the state (3, 2) and action 'Up'.

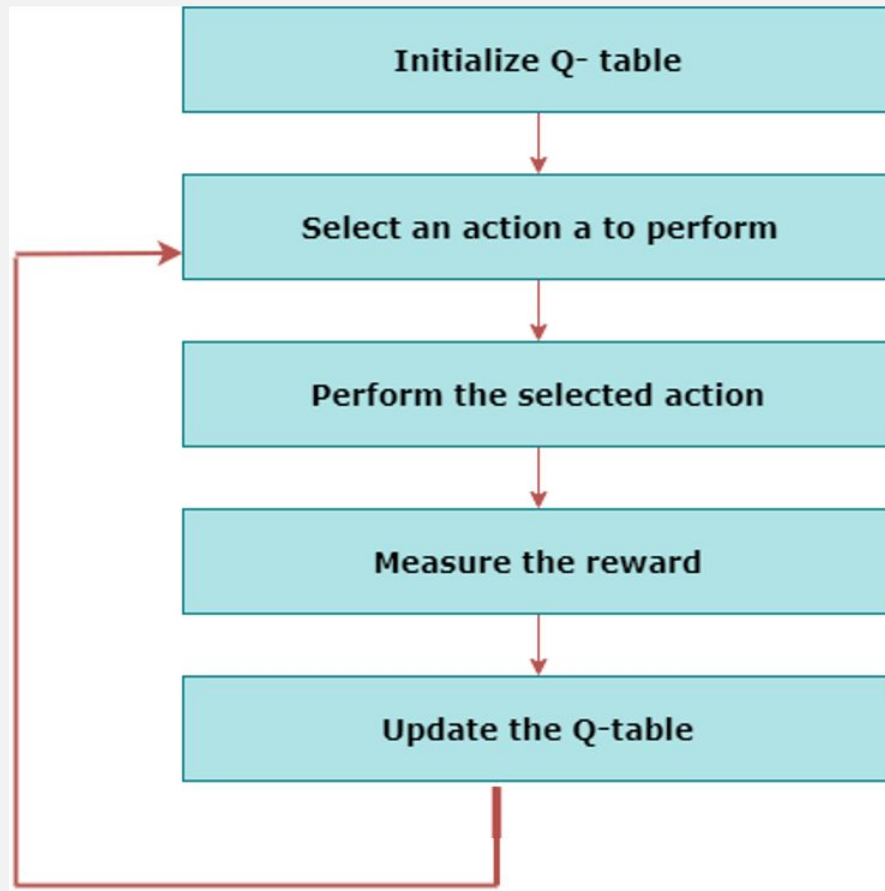
	Left	Right	Up	Down
(1,1)	0	0	0	0
(1,2)	0	0	0	0
(1,3)	0	0	0	0
(2,1)	0	0	0	0
(2,2)	0	0	0	0
(2,3)	0	0	0	0
(3,1)	0	0	0	0
(3,2)	0	0	0	0
(3,3)	0	0	0	0



- Q-learning finds the Optimal policy by learning the optimal Q-values for each state-action pair.
- Each entry in the Q-table represents the expected future reward (the Q-value) the agent will get if it takes a specific action in a given state.
- Initially, the agent randomly picks actions
- But as the agent interacts with the environment, it learns which actions are better, based on rewards that it obtains.
- It uses this experience to incrementally update the Q values. The goal is to learn the Q-values that lead to the most long-term rewards.

Q-learning – Off-policy model free RL

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$



From the current state, agent **choose/select** an action to execute using **ϵ -greedy** policy

Choose a random number: At each decision point, the algorithm generates a random number between 0 and 1.

- If the random number is less than epsilon (ϵ): The agent **explores** and chooses a random action.
- If the random number is greater than or equal to epsilon (ϵ): The agent **exploits** and chooses the action with the highest Q-value in the current state.